

Analysis of Algorithms Laboratory

Lab: Master Theorem, Merge Sort, and Quick Sort

Aispur P. Santiago

School of Mathematical and Computational Sciences

Yachay Tech University

San Miguel de Urcuquí - Imbabura, Ecuador

March 29, 2026

1 Introduction

This lab includes: (1) solving recurrences with the Master Theorem (paper work), (2) implementing Merge Sort in C++, (3) checking if Merge Sort is stable with an example, (4) implementing Quick Sort and testing with random lists, and (5) running empirical analysis for Quick Sort and comparing with the mathematical analysis.

2 Problem 3: Stability of Merge Sort

2.1 Definition of Stability

A sorting algorithm is **stable** if equal keys keep their relative order after sorting. If two elements have the same value and one appears first in the input, it must also appear first in the output.

2.2 Experiment and Example

To test stability, the input can be represented as pairs (`value`, `label`). The label is the original identity of the element.

Example input:

(2,A) (1,X) (2,B) (2,C) (1,Y)

Sorted by value:

(1,X) (1,Y) (2,A) (2,B) (2,C)

The three elements with value 2 stay in the same order: A then B then C. This matches the behavior of the Merge Sort implementation when the merge step selects from the left subarray first on ties (comparison $\leq$). For that reason, the implemented Merge Sort is stable.

2.3 Program Test

A Merge Sort test run with $n = 10$ produced:

Unsorted:

69 80 37 94 28 70 42 37 5 87

Sorted:

5 28 37 37 42 69 70 80 87 94

3 Problem 5: Empirical Analysis of Quick Sort

3.1 Mathematical Analysis

Quick Sort depends on partition balance:

- Average case time: $O(n \log n)$
- Worst case time: $O(n^2)$ (unbalanced partitions)
- Space: $O(\log n)$ average recursion depth, $O(n)$ worst case

3.2 Experiment Setup

Quick Sort was tested with randomly generated lists. Execution time was measured with `<chrono>`. The program prints the unsorted and sorted lists when $n \leq 50$.

3.3 Results

A sample run with $n = 10$ produced the output shown below, with measured time.

Quick Sort time: 0.0009 ms

Table 1 reports the measured time from the run.

Table 1: Quick Sort empirical timing result (sample run)

n	Time (ms)
10	0.0009

3.4 Discussion: Does it match the math?

With random input, Quick Sort is expected to follow $O(n \log n)$ behavior on average. A single small run ($n = 10$) is not enough to observe the growth trend clearly, because the execution time is extremely small. To compare properly with the mathematical analysis, the same program should be executed for larger values of n (for example 10^4 to 10^6) and the times recorded in a table. For large random lists, the growth should be close to $O(n \log n)$, while for already sorted input with a poor pivot choice it can approach $O(n^2)$.

Repository

Full source code available at: [Aispur Santiago - Analysis of Algorithms, GitHub Repository](#)

Master Theorem

• For a general recurrence $T(n) = aT(n/b) + f(n)$ where $f(n) \in O(n^d)$

- If $a < b^d \Rightarrow T(n) \in O(n^d)$
- If $a = b^d \Rightarrow T(n) \in O(n^d \log n)$
- If $a > b^d \Rightarrow T(n) \in O(n^{\log_b a})$

Algo - Lab 06

1. Solve the following recurrences using the Master Theorem.

4.1 $T(n) = 4T(n/2) + n, T(1) = 1$

• $a = 4$

• $b = 2$

• $f(n) = n$

* Step 2 Compute the critical term

$$n^{\log_b a} = n^{\log_2 4}$$

$$n^{\log_2 4} = n^2$$

* Step 3 Compare $f(n)$ with $n^{\log_b a}$

• $f(n) = n$

• $n^{\log_b a} = n^2$

$$n = O(n^{2-1}) \Rightarrow f(n) = O(n^{\log_b a - \epsilon})$$

* So:

$$T(n) = \Theta(n^{\log_b a}) = \Theta(n^2)$$

$$T(n) = \Theta(n^2)$$

4.2 $T(n) = 4T(n/2) + n^2$, $T(1) = 1$
 • Master theorem form

• $a = 4$

• $b = 2$

• $f(n) = n^2$

• Critical term:
 $n \log_b^a = n \log_2^4 = n^2$

• Compare $f(n)$ with $n \log_b^a$

• $f(n) = n^2$

• $n \log_b^a = n^2$

• $f(n) = \Theta(n \log_b^a)$

$T(n) = \Theta(n \log_b^a \log n) = \Theta(n^2 \log n)$

• $T(n) = \Theta(n^2 \log n)$

4.3 $T(n) = 4T(n/2) + n^3$, $T(1) = 1$

$a = 4$

$b = 2$

$f(n) = n^3$

• Critical term
 $n \log_b^a = n \log_2^4 = n^2$

• Compare $f(n)$ with $n \log_b^a = n^2$, since $n^2 < n^3$, then:

$T(n) = O(n^3)$

• $a f(n/b) \leq c f(n)$, $\forall c < 1$

$= 4 \left(f\left(\frac{n}{2}\right) \right) = 4 \left(\frac{n}{2} \right)^3 = \frac{n^3}{2} \leq c n^3$ // complies